

Q.1 Explain Extreme Programming (XP)

Extreme Programming (XP) is an agile software development methodology that aims to improve the quality and responsiveness of software development projects. The core values of Extreme Programming are communication, simplicity, feedback, and courage. These values guide the development team in their decision-making process and interactions with each other and stakeholders.

XP follows several key principles:

- 1.**Incremental Development**:** XP advocates for iterative and incremental development. The project is divided into short iterations, typically lasting one to three weeks, where small pieces of functionality are developed and delivered. Each iteration builds upon the previous one, allowing for continuous improvement and adaptation to changing requirements.
- 2.**Customer Involvement**:** Customers or stakeholders are actively involved throughout the development process. They work closely with the development team to define and prioritize user stories (requirements) and provide constant feedback on the product's progress. This collaboration ensures that the software meets the customer's needs effectively.
- 3.**Embracing Change**:** XP recognizes that change is inevitable in software development. Instead of resisting change, XP embraces it by allowing for flexibility and adaptation. The development team is prepared to modify requirements, code, and design as new information emerges. Frequent communication and feedback loops help in accommodating changes smoothly.
- 4.**Small, Empowered Teams**:** XP promotes small, self-organizing teams that are cross-functional and capable of delivering working software. Team members collaborate closely, communicate frequently, and share responsibilities. This approach fosters a sense of ownership, accountability, and collective decision-making within the team.
- 5.**Continuous Integration**:** XP emphasizes continuous integration, where code changes from multiple developers are integrated and tested frequently. The team integrates their work multiple times a day, ensuring that the software remains stable and functional at all times. This practice reduces integration issues and enables rapid feedback on any problems that arise.
- 6.**Test-Driven Development (TDD)**:** Test-driven development is a fundamental practice in XP. Developers write automated tests before writing the actual code. These tests serve as executable specifications and help drive the development process. TDD ensures that code is thoroughly tested, promotes better design, and enables faster refactoring.
- 7.**Refactoring**:** XP encourages refactoring, which involves improving the internal structure of the code without changing its external behavior. Refactoring ensures that the code remains clean, maintainable, and adaptable. With continuous integration and automated tests, developers can confidently refactor code without introducing regressions.
- 8.**Pair Programming**:** Pair programming is a practice in which two developers work together on the same task, with one person actively writing code (the driver) and the other providing feedback and thinking strategically (the navigator). Pair programming improves code quality, knowledge sharing, and collaboration within the team.

Q.4 Explain feature driven development.

Feature-Driven Development (FDD) is an agile software development methodology that focuses on building software incrementally based on individual features or functional requirements. It provides a framework for managing and organizing software development projects, particularly in large and complex systems.

FDD follows a five-step iterative and incremental process:

- 1.**Develop an Overall Model**:** The project begins by developing an overall domain model that represents the key aspects of the system. This model is created collaboratively by the development team, domain experts, and stakeholders. The model captures the essential entities, relationships, and business processes of the system.
- 2.**Build a Feature List**:** The next step involves identifying and defining a prioritized list of features or functional requirements based on the overall model. Each feature is a small, self-contained unit of work that can be completed within a few days to a couple of weeks.
- 3.**Plan by Feature**:** The development team selects a set of features to be worked on in each iteration. For each feature, they define the specific tasks, estimate the effort required, and assign them to individual developers. This planning process ensures a clear focus on completing features and delivering value incrementally.
- 4.**Design by Feature**:** Once the feature list and plan are established, the team proceeds to design each feature in detail. The design activities include identifying the classes, methods, and data structures required to implement the feature. The design is reviewed and refined by the team to ensure it aligns with the overall architecture and best practices.
- 5.**Build by Feature**:** With the design in place, developers work on implementing the features incrementally. Each developer takes responsibility for implementing specific features, ensuring that code is written according to the design and adheres to coding standards. The completed features are integrated and tested for functionality and quality.

Throughout the FDD process, regular progress meetings are held to review the status of each feature, address any issues or roadblocks, and make adjustments to the plan as necessary. FDD places a strong emphasis on collaboration, communication, and coordination within the development team.

Q.2 What Are 12 Manifesto Of Agile Development?

The Agile Manifesto is a foundational document that outlines the core values and principles of agile software development. The Agile Manifesto consists of the following twelve principles:

- 1.**Customer satisfaction through early and continuous software delivery**:** The primary focus of agile development is to deliver valuable software to customers as early as possible and to continuously iterate and improve upon it based on feedback.
- 2.**Embrace changing requirements**:** Agile development welcomes changing requirements, even in the late stages of development. The ability to adapt to changing customer needs and market conditions is considered essential.
- 3.**Frequent delivery of working software**:** Agile teams aim to deliver functional software in short, regular intervals, typically in iterations or sprints. This allows for continuous feedback, reduces risk, and enables early realization of business value.
- 4.**Collaboration between business stakeholders and developers**:** Agile development emphasizes close collaboration and active involvement of business stakeholders, customers, and developers throughout the project. This collaboration ensures that the software aligns with business goals and meets customer needs.
- 5.**Build projects around motivated individuals**:** Agile teams are built around self-motivated individuals who are trusted and empowered to make decisions. The focus is on creating an environment that promotes creativity, innovation, and personal growth.
- 6.**Face-to-face communication**:** Agile development values face-to-face communication as the most effective means of conveying information within a team. Direct and frequent communication fosters better understanding, reduces misunderstandings, and promotes collaboration.
- 7.**Working software as the primary measure of progress**:** The progress of an agile project is primarily measured by the functionality and quality of the working software delivered, rather than by documentation, plans, or other artifacts.
- 8.**Sustainable development pace**:** Agile teams strive for a sustainable development pace by maintaining a constant and manageable workload. This helps prevent burnout and enables the team to maintain a consistent and predictable delivery rhythm.
- 9.**Technical excellence and good design**:** Agile teams prioritize technical excellence and good design practices. They emphasize the importance of clean, maintainable code, as well as continuous refactoring and improvement of the software.
- 10.**Simplicity**:** Agile development promotes simplicity in both software design and processes. Unnecessary complexity is avoided, and the focus is on delivering the most valuable features with the least amount of complexity.
- 11.**Self-organizing teams**:** Agile teams are self-organizing, meaning they have the autonomy to determine how to best achieve their goals. Team members collaborate, make decisions collectively, and distribute work based on their skills and expertise.
- 12.**Regular reflections and adjustments**:** Agile teams regularly reflect on their work and processes, seeking opportunities for improvement. They adapt their approach based on feedback, learn from their experiences, and continuously refine their practices.

Q.5 Prepare SRS in IEEF Format for the systems.

****Software Requirements Specification (SRS)****

****1.1. Introduction****

1.1 Purpose

The purpose of this document is to define the requirements for the development of the [System Name]. It outlines the features, functionalities, and constraints of the system to be developed.

1.2 Scope

The [System Name] is a [brief description of the system's purpose and context]. It will provide [main functions and capabilities] to [end users or stakeholders].

1.3 Definitions, Acronyms, and Abbreviations

[Provide a list of definitions, acronyms, and abbreviations used in the document.]

****2. Overall Description****

2.1 Product Perspective

2.1.1 System Interfaces
[Describe the interfaces of the system with external entities such as hardware, software, or other systems.]

2.1.2 User Interfaces

[Describe the user interfaces, including screens, menus, and interactions.]

2.1.3 Hardware Interfaces

[Specify any hardware interfaces required by the system.]

2.1.4 Software Interfaces

[Specify any software interfaces required by the system.]

2.2 Product Functions

[Describe the main functions and features of the system.]

2.3 User Characteristics

[Describe the characteristics of the intended users or stakeholders of the system.]

2.4 Constraints

[Specify any constraints or limitations that affect the design or implementation of the system.]

****3. Specific Requirements****

3.1 Functional Requirements

3.1.1 [Requirement ID]: [Description of the requirement]

[Provide a detailed description of each functional requirement, including input, processing, and output.]

3.2 Non-functional Requirements

3.2.1 Performance

[Specify any performance-related requirements, such as response time, throughput, or capacity.]

3.2.2 Security

[Specify any security-related requirements, such as authentication, authorization, or data encryption.]

3.2.3 Reliability

[Specify any reliability-related requirements, such as availability, fault tolerance, or error handling.]

3.2.4 Usability

[Specify any usability-related requirements, such as user interface design, accessibility, or documentation.]

3.2.5 Maintainability

[Specify any maintainability-related requirements, such as modularity, extensibility, or ease of maintenance.]

****4. Appendices****

[Include any additional information, diagrams, or references that support the SRS.]

Q.3 Write a Difference Between Agile And Scrum ?

Agile	Scrum
Agile is a methodology	Scrum is used to implement Agile
In Agile, leadership plays a significant role.	Scrum fosters a cross-functional, self-organizing team.
Simple design and execution.	Innovative and creative design and execution.
All the tasks are managed and taken care of by the project head.	Any issues or challenges are addressed by the entire team.
Customer satisfaction is the topmost priority.	Philosophy of Scrum-based processes is- Empirical Process Control.
Face to face communication is emphasized to achieve the goal.	The key focus is in delivering Maximum business value.
The fundamental measure of progress-working software.	Working software is not considered as a fundamental measure of progress.
Collaborations and daily in-person interactions are involved in Agile.	Daily stand up meetings defines the responsibilities of various scrum roles.

Q.6 Diff between SSAD and OOAD?

SSAD	OOAD
1. SSAD evolved from Structured programming	OOAD evolved from Object Oriented Programming.
SSAD is process oriented.	OOAD is data oriented (objects).
SSAD breakdowns the system using DFDs(Data Flow Diagrams)	OOAD breaks down the system using Use cases.
SSAD the components of the system are derived from DFDs	OOAD the components are derived from class diagrams and other UML diagrams.
SSAD has defined steps, of planning , analysis, design and implementation	OOAD most of the times follows an iterative development. The system gets refined each time.
SSAD has a separation of system data and processes.	OOAD the system's data and process are encapsulated into objects.

Q.7 What is Data Abstraction Explain with example?

Data abstraction is a fundamental concept in programming and computer science that involves representing complex systems or entities by focusing on the essential characteristics and hiding unnecessary details. It allows developers to create abstract representations of real-world objects or concepts, making the code more modular, maintainable, and easier to understand.

In data abstraction, you define a clear boundary between the essential properties and behaviors of an object (known as the interface) and the underlying implementation details (known as the implementation). The interface exposes the necessary operations or methods that users can interact with, while the implementation remains hidden.

Let's take the example of a car to illustrate data abstraction. Suppose you are developing a software program that simulates a car. In this case, you can abstract the car as an object with certain properties and behaviors.

The interface of the car object might include methods such as:

- `start()`: Starts the car's engine.
- `stop()`: Stops the car's engine.
- `accelerate(speed)`: Accelerates the car to a specified speed.
- `brake()`: Applies the brakes to slow down or stop the car.

These methods represent the essential actions that users of the car object need to interact with. Users don't need to know the internal details of how the engine works or how the acceleration is achieved; they only need to know how to start, stop, accelerate, and brake the car.

The implementation of the car object, which is hidden from the users, contains the detailed code and logic that performs the actions defined in the interface. It may include components such as the engine, transmission system, braking system, and other internal mechanisms that work together to make the car function.

By abstracting the car object, you can create multiple instances of cars, each with its own independent state and behavior. Users can interact with these car objects through the defined interface, without worrying about the complexities of the internal implementation.

Q.8 short notes on waterfall model,Rad Model,prototype model,spiral model

1. **Waterfall Model**:

The Waterfall model is a linear sequential software development approach where progress flows steadily downwards like a waterfall. It consists of distinct phases, including requirements gathering, system design, implementation, testing, deployment, and maintenance. Each phase is completed in sequence, and progress moves forward only when the previous phase is completed. It lacks flexibility for accommodating changes once a phase is finished.

2. **RAD (Rapid Application Development) Model**:

The RAD model is an iterative and incremental software development methodology focused on delivering quick results. It emphasizes rapid prototyping, user feedback, and active user involvement. RAD breaks the project into small modules or components, allowing for parallel development and integration. It reduces the time between initial concept and final product delivery, making it suitable for projects with tight deadlines.

3. **Prototype Model**:

The Prototype model involves the creation of an initial working model of the software to gather user feedback and refine requirements. It helps stakeholders visualize and understand the system early in the development process. The prototype is developed iteratively, and feedback is gathered to refine the design and requirements. Once the prototype is approved, it serves as the basis for the full-scale development.

4. **Spiral Model**:

The Spiral model combines elements of both iterative and waterfall models. It incorporates risk analysis and mitigation throughout the development process. The model consists of multiple iterations called spirals, with each spiral representing a phase of the software development life cycle. The project progresses through each spiral, and at the end of each iteration, the software is evaluated, risks are assessed, and decisions are made regarding the next iteration.

Q.11 Explain Phases Of Requirement Engineering

In Object-Oriented Systems Engineering (OOSE), the phases of Requirement Engineering focus on eliciting, analyzing, and documenting the requirements of a software system using an object-oriented approach. These phases help capture the system's functionality, behavior, and constraints. Let's discuss the phases of Requirement Engineering in OOSE:

1. ****Requirements Elicitation****: This phase involves gathering information about the system's requirements from various stakeholders, including clients, end-users, domain experts, and other relevant parties. Techniques such as interviews, surveys, workshops, and brainstorming sessions are employed to collect requirements. The goal is to identify the system's scope, objectives, functionalities, and constraints.

2. ****Requirements Analysis****: In this phase, the elicited requirements are analyzed in detail to understand their impact on the system's design and behavior. The requirements are prioritized, clarified, and refined. Techniques like use case analysis, scenario-based analysis, and data analysis are employed to model the system's behavior, identify user interactions, and define functional and non-functional requirements.

3. ****Domain Modeling****: Domain modeling focuses on representing the concepts, entities, and relationships within the problem domain. It involves creating conceptual models using techniques such as UML class diagrams, activity diagrams, or domain-specific modeling languages. Domain models help in understanding the problem space, identifying key entities, and establishing relationships and associations between them.

4. ****Requirements Specification****: In this phase, the requirements are documented in a clear and structured manner. The requirements specification includes use case diagrams, use case descriptions, system requirements, functional and non-functional requirements, and any other relevant documentation. The specification serves as a formal reference for developers, testers, and other stakeholders to understand the system's requirements and design.

5. ****Requirements Validation****: Requirements validation ensures that the specified requirements accurately represent the stakeholders' needs and are feasible to implement. The validation process involves reviewing the requirements with stakeholders, seeking their feedback and approval, and conducting verification techniques such as walkthroughs, inspections, and prototyping. The goal is to identify and address any ambiguities, conflicts, or gaps in the requirements.

6. ****Requirements Management****: Requirements management focuses on managing changes to the requirements throughout the system's lifecycle. It involves establishing a change control process, documenting change requests, assessing their impact, and ensuring proper communication and coordination with stakeholders. Requirements management ensures that the requirements remain up-to-date, consistent, and aligned with the evolving needs of the system.

Q.9 Explain SDLC In Detail?

SDLC (Software Development Life Cycle) is a systematic process that guides the development of software applications from inception to deployment and maintenance. It encompasses a series of well-defined phases, activities, and deliverables to ensure the successful and efficient development of software. Let's explore each phase of the SDLC in detail:

1. ****Requirements Gathering****: In this initial phase, project stakeholders, including clients, users, and development teams, identify and document the software's functional and non-functional requirements. This involves gathering information, conducting interviews, and analyzing existing systems or processes. The outcome is a detailed and comprehensive requirements specification document.

2. ****System Analysis****: In this phase, the requirements are analyzed in-depth to understand the scope of the system and determine how it will meet the users' needs. The system analysis involves creating models, such as use case diagrams, data flow diagrams, and class diagrams, to represent the system's structure and behavior. The analysis results in a system design document that outlines the system's architecture and modules.

3. ****System Design****: In this phase, the system's architecture and detailed design are created based on the analysis. The design includes specifications for each module, data structures, algorithms, and user interfaces. It also defines the software's overall structure, such as the database schema, network architecture, and software components. The design phase provides a blueprint for the development process.

4. ****Coding and Implementation****: Once the design is complete, the development team starts writing code according to the specifications and design guidelines. This phase involves coding, unit testing, and integrating the different modules. Programming languages, frameworks, and development tools are used to create the software. The output of this phase is a functioning software system that meets the defined requirements.

5. ****Testing****: The testing phase is crucial for ensuring that the software functions correctly and meets the specified requirements. It involves various types of testing, such as unit testing, integration testing, system testing, and acceptance testing. Test cases are executed, and defects are identified and fixed. The goal is to verify the software's quality, functionality, performance, and security.

6. ****Deployment****: In this phase, the software is deployed to the production environment or made available to end-users. This includes activities like installation, configuration, and data migration. User training and documentation may also be provided to assist users in using the software effectively. Deployment may involve gradual rollouts or updates to ensure a smooth transition from the previous system.

7. ****Maintenance****: Once the software is deployed, it enters the maintenance phase. This phase involves monitoring the software's performance, addressing user feedback, fixing defects, and making updates or enhancements as required. Regular maintenance activities, such as bug fixes and software updates, are performed to ensure the software remains reliable, secure, and up-to-date.

Q.12 Oops Concepts In software engineering?

Object-oriented programming (OOP) concepts are fundamental principles in software engineering that facilitate the design, implementation, and maintenance of software systems. Here are some of the key OOP concepts:

1. ****Objects****: Objects are the basic building blocks of an object-oriented system. They represent real-world entities or concepts and encapsulate both data (attributes) and behavior (methods) related to those entities. Objects are instantiated from classes and interact with each other through message passing.

2. ****Classes****: Classes define the blueprint or template for creating objects. They encapsulate the common attributes and behaviors shared by a group of objects. Classes specify the structure and behavior of objects and serve as a blueprint for creating multiple instances of similar objects.

3. ****Inheritance****: Inheritance allows objects (derived or child classes) to inherit properties and behaviors from other objects (base or parent classes). It enables code reuse, promotes modularity, and facilitates the creation of hierarchical relationships between classes.

4. ****Polymorphism****: Polymorphism refers to the ability of objects to exhibit different forms or behaviors depending on the context. It allows objects of different classes to be treated interchangeably, as long as they share a common interface or inheritance hierarchy. Polymorphism enables code flexibility, extensibility, and dynamic dispatch of method calls.

5. ****Encapsulation****: Encapsulation is the process of bundling data and methods into a single unit called an object. It hides the internal implementation details of an object and exposes only the necessary interfaces for interacting with the object. Encapsulation promotes information hiding, enhances code maintainability, and protects data integrity.

6. ****Abstraction****: Abstraction refers to the process of simplifying complex systems by focusing on the essential aspects while hiding unnecessary details. It allows developers to create abstract representations of real-world entities or concepts. Abstraction provides a high-level view of the system, emphasizing the essential features and behaviors.

7. ****Association****: Association represents a relationship between two or more objects. It defines how objects are connected and interact with each other. Associations can be one-to-one, one-to-many, or many-to-many, and they establish the connections and dependencies between objects.

8. ****Aggregation****: Aggregation represents a special form of association where one object is composed of other objects. It implies a "whole-part" relationship, where the lifetime of the composed objects can exist independently of the aggregate object.

9. ****Composition****: Composition is a stronger form of aggregation, where the composed objects are tightly dependent on the existence of the aggregate object. The composed objects cannot exist without the aggregate, and they are often created and destroyed together.

10. ****Interfaces****: Interfaces define a contract or set of methods that a class must implement. They specify the behavior that objects of the class should adhere to. Interfaces provide a way to achieve polymorphism, allowing objects of different classes to be treated uniformly based on the shared interface.

Q.10 Discuss About Class And Object?

In Object-Oriented Systems Engineering (OOSE), the concepts of class and object are fundamental to modeling and designing software systems. OOSE extends the principles of object-oriented programming (OOP) to the analysis, design, and implementation of large-scale systems. Let's discuss how class and object are relevant in OOSE:

Class:

In OOSE, a class is a fundamental building block that represents a template or blueprint for creating objects. A class encapsulates the attributes and behaviors of a particular entity or concept within the system being developed. It defines the structure, data, and methods that objects of that class will possess.

Classes in OOSE have additional significance beyond traditional OOP. They serve as a means to model system requirements, constraints, and relationships, helping to capture the system's structure and behavior. Classes are typically represented using Unified Modeling Language (UML) class diagrams, which visually depict the class's attributes, methods, and relationships with other classes.

A class in OOSE can have various elements, including:

- ****Attributes****: Attributes represent the data or state associated with objects of the class. They define the characteristics, properties, or information that objects of that class possess.

- ****Methods****: Methods are the operations or functions defined within a class that describe the behavior or actions that objects of the class can perform.

Object:

An object, in the context of OOSE, is an instance or occurrence of a class. It represents a specific entity or concept within the system. Objects are concrete manifestations of the class, with their own unique state and behavior.

Objects are created from classes and are characterized by their attribute values. Each object has its own set of attribute values, and it can invoke the methods defined in the corresponding class to perform actions or interact with other objects.

In OOSE, objects are used to represent real-world entities, system components, or any other relevant concept within the system being modeled. Objects are connected through relationships and interactions, forming a complex network of interconnected entities within the system.

By creating objects and defining their relationships and interactions, OOSE enables the modeling and analysis of complex system behavior, structure, and requirements. Objects in OOSE help in understanding how entities interact, collaborate, and fulfill their roles within the system.

Q.13 .Discuss About Types Of UML Diagrams in short?

Unified Modeling Language (UML) offers several types of diagrams to visually represent different aspects of software systems and facilitate communication among stakeholders. Here are some of the key types of UML diagrams:

1. ****Class Diagram****: Class diagrams depict the static structure of a system by showing classes, their attributes, methods, and relationships with other classes. They highlight the system's entities, their properties, and the associations between them.

2. ****Use Case Diagram****: Use case diagrams depict the system's functionality from a user's perspective. They identify the actors (users, external systems) and the use cases (interactions or actions) they perform. Use case diagrams show how actors and use cases relate and help identify system requirements.

3. ****Sequence Diagram****: Sequence diagrams illustrate the dynamic behavior of a system by showing the interactions between objects over time. They represent the flow of messages between objects in a chronological order, emphasizing the sequence of actions and the collaboration between objects.

4. ****Activity Diagram****: Activity diagrams depict the flow of activities or actions within a system or a specific process. They show the steps, decisions, branching, and concurrency of activities, making them useful for modeling business processes, workflows, or system behaviors.

5. ****State Machine Diagram****: State machine diagrams represent the different states that an object can be in and how it transitions between states based on events. They illustrate the behavior of objects, capturing the conditions, triggers, and actions associated with state changes.

6. ****Component Diagram****: Component diagrams provide a high-level view of the system's physical and logical components, such as modules, libraries, or software units. They depict the relationships between components and how they interact to form the system.

7. ****Deployment Diagram****: Deployment diagrams show the physical deployment of software components onto hardware nodes or execution environments. They depict the hardware infrastructure, software artifacts, and their associations, helping visualize the system's deployment architecture.

8. ****Package Diagram****: Package diagrams group related elements, such as classes or components, into packages. They illustrate the organization and dependencies between packages, providing a structural overview of the system's modules and their relationships.

Q.14 What Are the 8 Golden rules for design?

The "8 Golden Rules for Design" are a set of principles that help guide the design process and promote the creation of effective and visually appealing designs. These rules are widely recognized in the field of design and can be applied across various disciplines, including graphic design, user interface design, and web design. Here are the 8 Golden Rules:

1. **Consistency:** Maintain consistency in design elements such as colors, typography, layout, and interactions throughout the design. Consistency provides a sense of cohesion and makes the design more intuitive and user-friendly.

2. **Contrast:** Create contrast by using different elements such as colors, sizes, or shapes to make important elements stand out and draw attention. Contrast adds visual interest and helps guide the user's focus.

3. **Hierarchy:** Establish a clear visual hierarchy by organizing and prioritizing design elements based on their importance. Proper hierarchy ensures that users can easily scan and understand the content and interface.

4. **Alignment:** Use alignment to create a sense of order and unity in the design. Align elements to a grid or other visual cues to establish relationships and improve readability.

5. **Repetition:** Repeat design elements such as colors, typography, or icons to create a cohesive and unified visual identity. Repetition helps establish consistency and reinforces the design's overall aesthetic.

6. **Proximity:** Group related elements together to visually connect them and establish their relationship. Proximity improves the organization and readability of the design, allowing users to quickly understand the connections between different elements.

7. **Simplicity:** Strive for simplicity by removing unnecessary elements and visual clutter. Keep the design clean and minimal, focusing on the core message or functionality. Simple designs are often more intuitive and user-friendly.

8. **Accessibility:** Design with accessibility in mind to ensure that the design can be easily understood and used by a wide range of users, including those with disabilities. Consider factors such as color contrast, text size, and navigation options to make the design inclusive and accessible to all.

Q.17 What is DSDM Method, Explain In Details?

DSDM (Dynamic Systems Development Method) is an Agile project delivery framework that provides an iterative and incremental approach to software development. It is characterized by its focus on delivering business value, active user involvement, and continuous collaboration among stakeholders. DSDM provides guidelines, principles, and best practices for managing and delivering projects in an Agile manner.

Here are the key components and principles of DSDM:

1. **Iterative Development:** DSDM uses time-boxed iterations, known as "timeboxes," to deliver working software increments. Each timebox typically lasts 2-6 weeks and focuses on delivering a prioritized set of features. DSDM follows the "MoSCoW" prioritization technique, which categorizes requirements as Must have, Should have, Could have, and Won't have.

2. **Focus on Business Needs:** DSDM places a strong emphasis on delivering business value. The project team collaborates closely with business stakeholders to identify and prioritize requirements that align with the organization's strategic goals. DSDM ensures that delivered software meets the needs of the business and provides tangible benefits.

3. **Active User Involvement:** DSDM recognizes the critical role of users and promotes their active involvement throughout the project. Users are engaged in requirements gathering, prioritization, and validation, ensuring that the software meets their needs. User representatives are part of the project team, facilitating continuous feedback and decision-making.

4. **Collaboration and Communication:** DSDM emphasizes open and collaborative communication among all project stakeholders. Regular workshops, daily stand-ups, and frequent feedback loops facilitate effective communication and help ensure that the team is aligned with business goals. Face-to-face interactions are encouraged to foster better understanding and quick problem resolution.

5. **Timeboxing:** DSDM employs timeboxing to manage project schedules and ensure focused and timely delivery. Timeboxes are fixed-length iterations with clear objectives and deliverables. The team commits to delivering the agreed-upon features within each timebox, and any unfinished work is re-prioritized for subsequent iterations.

6. **Iterative Planning and Incremental Delivery:** DSDM adopts an iterative and incremental approach to planning and delivery. The project plan is continuously refined and adjusted based on feedback and changing requirements. Frequent delivery of working software enables stakeholders to provide early feedback and ensures that the delivered product meets evolving needs.

7. **Prototyping and Iterative Development:** DSDM encourages prototyping to validate requirements and obtain early feedback. Prototypes are used to gather user feedback, refine requirements, and demonstrate the system's functionality. The system is developed incrementally, with regular feedback loops and demonstrations to stakeholders.

8. **Continuous Integration and Testing:** DSDM emphasizes continuous integration, where code changes from different team members are regularly integrated into a shared codebase. Continuous testing is conducted to ensure quality and validate requirements. Automated testing is commonly employed to streamline the testing process.

Q.15 What is Web Engineering?

Web Engineering takes into consideration various factors unique to web development, such as the distributed nature of the web, scalability, security, performance, usability, and interoperability. It aims to create web applications that are reliable, user-friendly, secure, and capable of handling a large number of concurrent users.

The key aspects of Web Engineering include:

1. **Requirements Engineering:** Gathering and analyzing user requirements for the web application, considering the specific needs of stakeholders, target users, and the business context.

2. **Web Design:** Creating a user-centered design that focuses on the visual aesthetics, usability, and interaction design of the web application. This involves designing the user interface, navigation structure, and information architecture.

3. **Web Development:** Implementing the web application based on the design specifications and requirements. This includes coding the front-end (client-side) components using HTML, CSS, JavaScript, and other relevant technologies, as well as developing the back-end (server-side) logic using programming languages like Python, Java, or PHP.

4. **Web Testing:** Conducting various types of testing, such as functionality testing, performance testing, security testing, and compatibility testing, to ensure the quality and reliability of the web application.

5. **Web Deployment:** Deploying the web application to the hosting environment, configuring the servers, databases, and network infrastructure, and ensuring the application is accessible to users over the web.

6. **Web Maintenance:** Monitoring and maintaining the web application after deployment, addressing issues, applying updates, and continuously improving its performance, security, and functionality.

Web Engineering also considers important aspects such as search engine optimization (SEO), web analytics, accessibility compliance, responsive design for multiple devices, and compliance with web standards and best practices.

Q.18 Short note on ASD

ASD (Adaptive Software Development) is an Agile software development approach that focuses on managing complex and evolving projects. Here are some key characteristics and principles of ASD:

1. **Speculate:** ASD acknowledges that requirements and circumstances can change during the project. Rather than trying to predict and plan for every detail upfront, ASD encourages speculation, where the team creates an initial plan based on available information and continuously adjusts it as new insights emerge.

2. **Collaborate:** Collaboration is crucial in ASD. The team members, including developers, stakeholders, and customers, work closely together to understand the requirements, identify risks, and make informed decisions. Continuous communication and shared responsibility foster a collaborative environment.

3. **Learn:** ASD promotes a learning mindset. The team continuously gathers feedback, analyzes it, and applies the learnings to improve the development process. Learning from past experiences helps to enhance future iterations and deliver better software.

4. **Deliver Incrementally:** ASD advocates delivering software in small increments rather than waiting for a complete product at the end. This allows for early value delivery, quicker feedback loops, and the ability to adapt based on stakeholder inputs.

5. **Adapt:** ASD embraces change and accepts that requirements will evolve. It emphasizes the ability to adapt to changing circumstances, leveraging incremental development, frequent inspections, and regular feedback to accommodate changes and ensure that the software meets the evolving needs of stakeholders.

6. **Focus on Quality:** ASD places a strong emphasis on delivering high-quality software. Through continuous testing, inspections, and reviews, the team ensures that the product meets the desired quality standards. Quality assurance is an integral part of the development process.

Q. 16 Explain Agile Process Model ?

Here are the key characteristics and components of the Agile process model:

1. **Iterative Development:** Agile projects are divided into small iterations, typically referred to as "sprints," which are time-boxed periods (usually 1-4 weeks) where a set of features or user stories are developed, tested, and delivered. Each iteration adds value to the product, and subsequent iterations build upon the work done in previous iterations.

2. **Cross-functional Teams:** Agile teams are self-organizing and cross-functional, consisting of members with different skills and expertise necessary to deliver a complete product increment. The team collaborates closely, promoting communication, knowledge sharing, and collective decision-making.

3. **Product Backlog:** The product backlog is a prioritized list of user stories, features, or requirements that represent the desired functionality of the software product. It is dynamic and can evolve throughout the project as new requirements emerge or existing ones change. The product owner is responsible for managing and prioritizing the backlog.

4. **Sprint Planning:** At the beginning of each sprint, the development team, product owner, and scrum master collaborate to select and commit to a set of items from the product backlog that can be completed within the sprint. The team breaks down the selected items into smaller tasks, estimates effort, and creates a sprint backlog.

5. **Daily Stand-up Meetings:** Daily stand-up meetings, also known as daily scrums, are short meetings (usually 15 minutes) where the team gathers to discuss progress, share updates, and identify any obstacles. It promotes transparency, coordination, and quick problem-solving.

6. **Continuous Integration and Testing:** Agile teams emphasize continuous integration, where the code changes made by individual team members are frequently integrated into a shared repository. Automated testing is also a crucial aspect of Agile development, enabling the team to catch and fix issues early.

7. **Sprint Review and Retrospective:** At the end of each sprint, the team conducts a sprint review meeting to demonstrate the completed work to stakeholders and gather feedback. This feedback is then used to refine the product backlog. Additionally, a sprint retrospective is held to reflect on the team's performance and identify areas for improvement.

8. **Adaptive Planning and Flexibility:** The Agile model embraces change and allows for flexibility in responding to evolving requirements and customer feedback. The team regularly adapts the project plan based on new information and adjusts priorities to deliver the highest value features.

Q.19 Diff between waterfall model and spiral model?

	Waterfall Model	Spiral Model
Model Complexity	Simple, linear sequence of phases	Complex, iterative model; many integrated tasks
Management	Document driven	Risk driven
Quality Control	Natural milestone after each phase	Continuous evaluation, integrated into the model
Customer Interaction	No	Prototypes are built and evaluated by customers in every iteration
Risk	High (late feedback)	Low (risk analysis is integrated in the model)
Usability	Small and/or low risk projects	Large projects

PWC-H06 11

Q.21 Difference between Functional and Non Functional Requirements ?

Functional Requirements	Non-functional Requirements
Describe what the system as a whole should do.	Describe the attributes of system quality and performance.
Cover all the functions that the software must perform.	Cover all aspects of good user experience.
Ensure all core functionality is well-performed.	Ensure users' needs are satisfied.
Easy to specify.	Difficult to specify.
They are tested first.	They are tested after functional testing.
What is tested: API testing, Functional testing of the whole system, integration, end to end testing, etc.	What is tested: Usability, Performance, Security, Stress testing, etc.
Type: Business rules, Administrative functions, Data management, Certification requirements, Authorization levels, etc.	Type: Availability, Scalability, Capacity, Reliability, Data integrity, etc.

Q.20 Explain Structure and contents of SRS?

The Structure and Contents of a Software Requirements Specification (SRS) may vary depending on the organization or project-specific requirements. However, a typical SRS document consists of the following sections:

1. **Introduction**: This section provides an overview of the SRS document, including its purpose, scope, and intended audience. It may also include a brief description of the system and its key features.
2. **Overall Description**: This section provides a high-level description of the system, its objectives, and the context in which it will be used. It may include information about the stakeholders, user characteristics, and any relevant assumptions or constraints.
3. **Specific Requirements**: This is the core section of the SRS that outlines the specific functional and non-functional requirements of the system. The requirements are typically organized into subsections for easier reference and understanding. Functional requirements describe the system's intended behavior, while non-functional requirements specify the system's quality attributes such as performance, security, reliability, and usability.
4. **External Interface Requirements**: This section details the interfaces between the system and external entities, such as other systems, hardware devices, or users. It includes information on the data formats, communication protocols, and interaction mechanisms.
5. **System Features**: This section provides a detailed description of the system's features or modules. It describes each feature's functionality, behavior, and any specific requirements associated with it.
6. **System Constraints**: Here, any constraints or limitations that affect the design or implementation of the system are documented. This may include technical, organizational, or regulatory constraints.
7. **User Documentation**: This section specifies the requirements for user documentation, such as user manuals, help guides, or online documentation. It outlines the content, format, and delivery mechanisms for the user documentation.
8. **Assumptions and Dependencies**: This section captures any assumptions made during the requirements gathering process and highlights any dependencies on external factors or systems.
9. **References**: This section provides a list of references used in the preparation of the SRS, such as relevant standards, specifications, or documentation.
10. **Appendices**: Appendices may include additional information that supports the understanding of the requirements, such as glossaries, diagrams, or supplementary documentation.

Q.22 Explain Rational Unified Process?

Here are the key components and characteristics of Rational Unified Process (RUP):

1. **Iterative and Incremental Development**: RUP follows an iterative and incremental development approach, where the project is divided into multiple iterations. Each iteration produces a working software increment, allowing for early value delivery and continuous feedback from stakeholders.
2. **Phases**: RUP defines a set of distinct phases that guide the project from inception to deployment and maintenance. The typical phases in RUP include Inception, Elaboration, Construction, and Transition. Each phase has specific goals, tasks, and deliverables, ensuring a structured and disciplined approach to development.
3. **Workflows**: RUP organizes activities and tasks into workflows that span the project lifecycle. Workflows represent distinct areas of development, such as business modeling, requirements analysis, design, implementation, testing, and deployment. Each workflow provides guidelines, artifacts, and techniques specific to that area.
4. **Artifacts**: RUP emphasizes the creation and management of various artifacts throughout the development process. Artifacts include documents, models, diagrams, prototypes, and code. Examples of artifacts in RUP include use case models, architectural designs, test plans, and user documentation. These artifacts capture and communicate information critical to the project's success.
5. **Roles and Responsibilities**: RUP defines specific roles and responsibilities for team members involved in the project. Roles may include the Project Manager, Business Analyst, Architect, Developer, Tester, and others. Each role has defined responsibilities and contributes to the project in their specific area of expertise.
6. **Risk Management**: RUP incorporates risk management practices throughout the project lifecycle. It encourages proactive identification, analysis, and mitigation of risks. Risk management helps teams address uncertainties and potential obstacles that may impact the project's success.
7. **Architecture-Centric Approach**: RUP emphasizes the importance of a well-defined and robust software architecture. It promotes architectural modeling, analysis, and validation to ensure that the system's structure and design align with the project goals and requirements. Architecture-centric development helps address quality attributes such as scalability, maintainability, and performance.
8. **Tool Support**: RUP supports the use of tools to facilitate development activities, collaboration, and documentation. Various software development tools can be integrated into the RUP framework to enhance productivity, automate tasks, and manage project artifacts.

Q.23 Roles of SDLC?

In the Software Development Life Cycle (SDLC), different roles are involved at various stages to ensure the successful development, implementation, and maintenance of a software system. These roles work collaboratively to execute tasks, make decisions, and contribute their expertise to deliver a high-quality software product. Here are some common roles in the SDLC:

1. **Project Manager**: The Project Manager oversees the entire software development project. They are responsible for planning, organizing, and coordinating resources, timelines, and deliverables. They ensure that the project stays on track, manages risks, and communicates with stakeholders.
2. **Business Analyst**: The Business Analyst acts as a bridge between the development team and stakeholders. They gather and analyze business requirements, elicit user needs, define system goals, and document functional and non-functional requirements. They facilitate communication and ensure that the software aligns with the business objectives.
3. **Software Architect**: The Software Architect designs the overall structure and architecture of the software system. They make high-level design decisions, define system components, and ensure that the system is scalable, maintainable, and robust. They collaborate with the development team to guide the implementation process.
4. **Software Developer/Programmer**: Software Developers/Programmers are responsible for coding and implementing the software according to the requirements and design specifications. They write code, perform unit testing, and collaborate with other team members to integrate their code into the system.
5. **Quality Assurance (QA) Engineer**: QA Engineers are responsible for ensuring the quality and reliability of the software. They create test plans, perform testing activities (such as functional testing, regression testing, and performance testing), identify and report defects, and validate that the software meets the defined requirements and standards.
6. **User Experience (UX) Designer**: The UX Designer focuses on creating an optimal user experience by designing the user interface, interaction flows, and visual aesthetics. They conduct user research, create wireframes and prototypes, and collaborate with stakeholders to ensure a user-friendly and intuitive design.
7. **Database Administrator (DBA)**: The DBA manages the database system and its administration. They design and maintain the database schema, ensure data integrity and security, optimize database performance, and manage backups and recovery procedures.
8. **System Administrator**: The System Administrator is responsible for configuring and managing the software environment, including servers, networks, and infrastructure. They install and maintain the software, handle system configurations, manage user access, and monitor system performance.